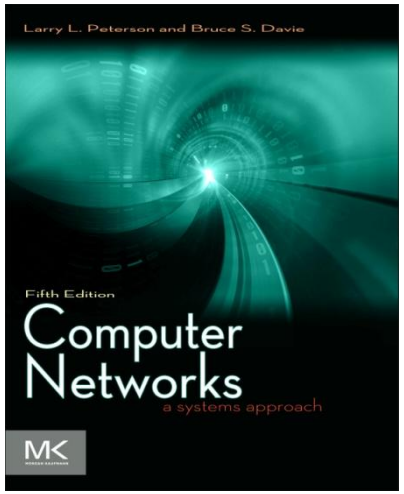


Chapter 6

Congestion Control and Resource Allocation

Problem

- Problem- Allocating Resources
  - How to *effectively* and *fairly* allocate resources among a collection of competing users?
  - Spans across the entire protocol stack

Chapter Outline

- Issues in Resource Allocation
- Queuing Disciplines
- TCP Congestion Control
- Congestion Avoidance Mechanism
- Quality of Service

Congestion Control and Resource Allocation

- Resources
  - Bandwidth of the links
  - Buffers at the routers and switches

- Packets contend at a router for the use of a link, with each contending packet placed in a queue waiting for its turn to be transmitted over the link

Congestion Control and Resource Allocation

- When too many packets are contending for the same link
  - The queue overflows
  - Packets experience higher end-to-end delay
  - Packets get dropped
    - Network is congested!- when long queues persist and drops become common

Congestion Control and Resource Allocation

- If the network takes active role in allocating resources
  - The congestion may be avoided
  - No need for congestion control
- But, allocating resources with precision is difficult
  - Resources are distributed throughout the network

Congestion Control and Resource Allocation

- On the other hand, let the sources send as much data as they want, then recover from the congestion when it occurs
- Easier approach but it can be disruptive as many packets may be discarded before congestion is controlled

Congestion Control and Resource Allocation

- In network elements
  - Various queuing disciplines used to control the order in which packets get transmitted
- At the hosts' end
  - The congestion control mechanism paces how fast sources are allowed to send packets

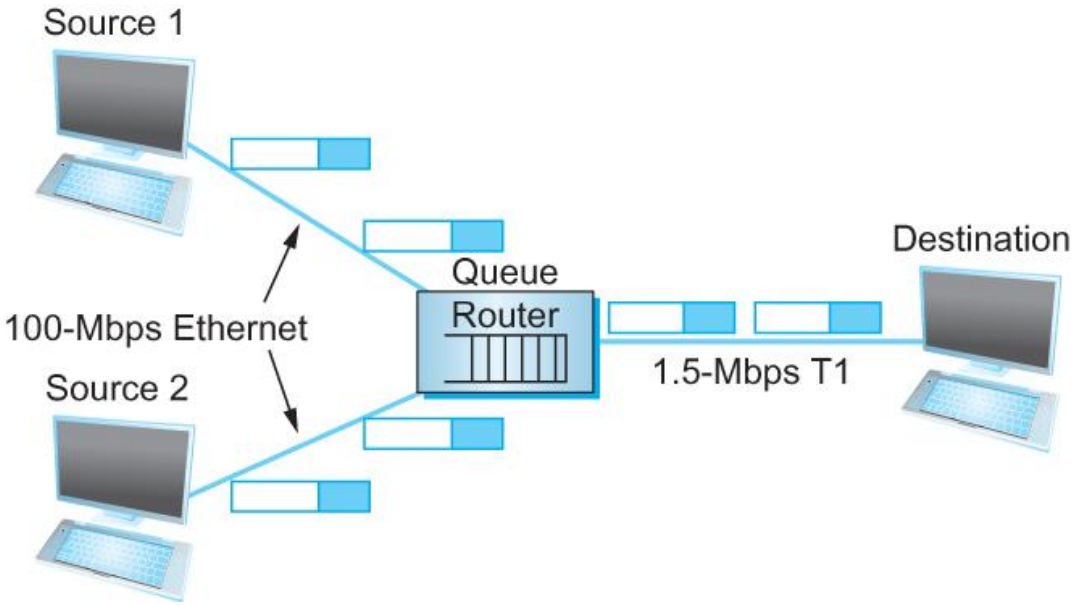
ISSUES IN RESOURCE ALLOCATION

Issues in Resource Allocation

- Network Model
  - Packet Switched Network
    - In packet-switched network (or internet) consisting of multiple links and switches (or routers),
    - A given source may have more than enough capacity on the immediate outgoing link to send a packet, but somewhere in the network, its packets encounter a link that is being used by many different traffic sources

Issues in Resource Allocation

- Network Model
  - Packet Switched Network



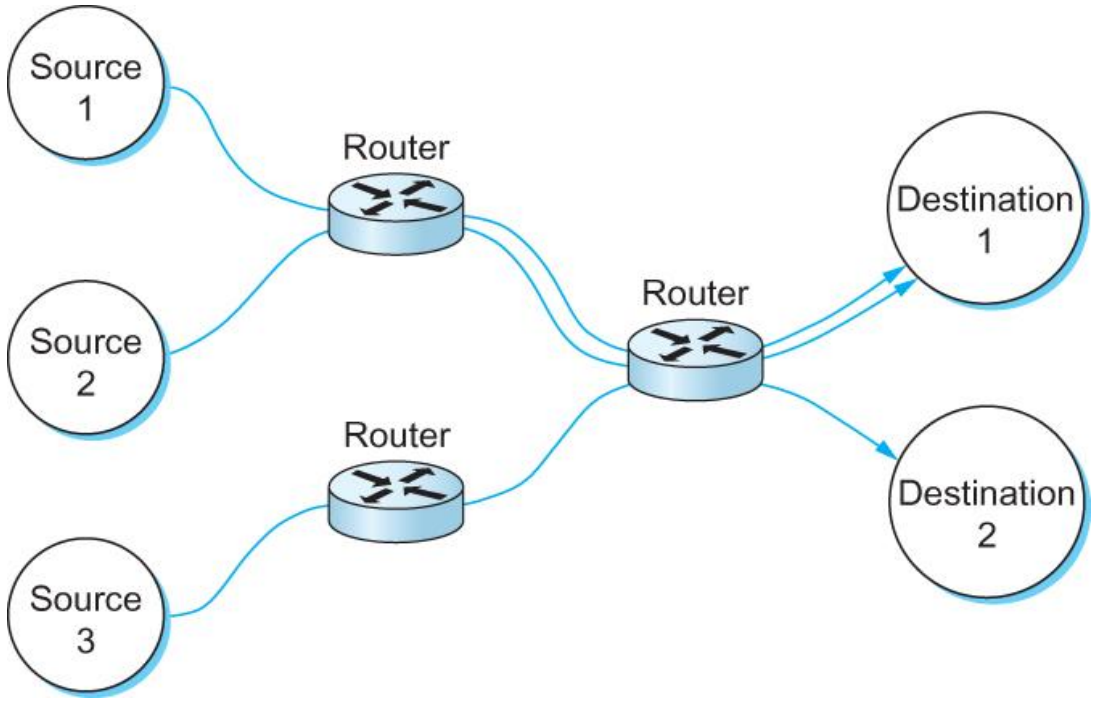
A potential bottleneck router.

Issues in Resource Allocation

- Network Model
  - Connectionless Flows
    - Assume that the network is connectionless, with any connection-oriented service implemented in the transport protocol that is running on the end hosts.
    - The datagrams are switched independently, but it is usually the case that a stream of datagrams between a particular pair of hosts flows through a particular set of routers

Issues in Resource Allocation

- Network Model
  - Connectionless Flows



Multiple flows passing through a set of routers

Issues in Resource Allocation

- Network Model
  - Connectionless Flows
    - One of the powers of the flow abstraction is that flows can be defined at different granularities.
    - For example, a flow can be host-to-host or process-to-process.
    - In the latter case, a flow is essentially the same as a channel.
    - The difference being a flow is visible to the routers inside the network, whereas a channel is an end-to-end abstraction.

Issues in Resource Allocation

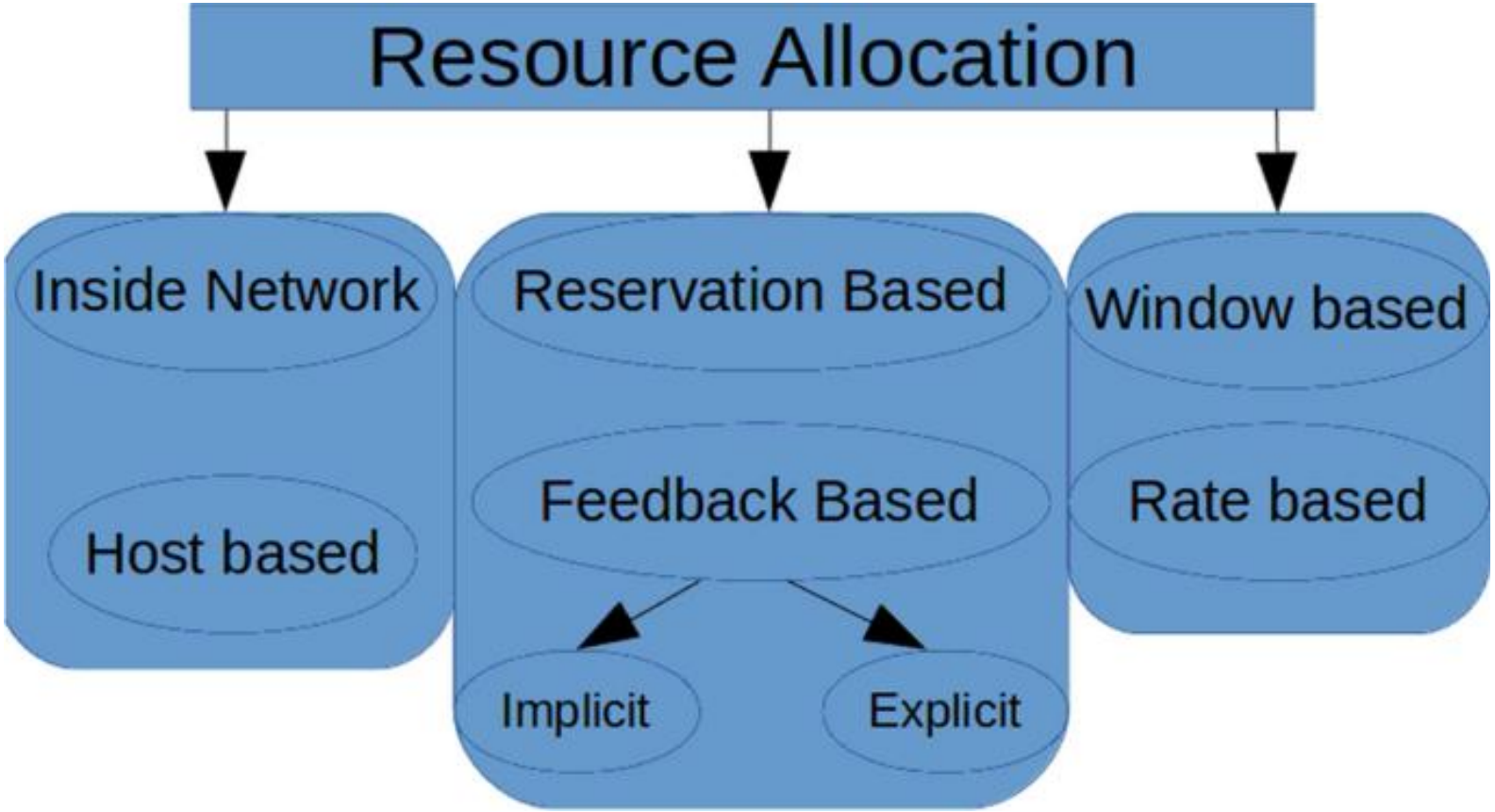
- Network Model
  - Connectionless Flows
    - Because multiple related packets flow through each router, it sometimes makes sense to maintain some state information for each flow. This state is sometimes called soft state.
    - The main difference between soft state and “hard” state is that soft state need not always be explicitly created and removed by signaling.

Issues in Resource Allocation

- Network Model
  - Connectionless Flows
    - Soft state represents a middle ground between a purely connectionless network and a purely connection-oriented network.
    - In general, when a packet happens to belong to a flow for which the router is currently maintaining soft state, then the router is better able to handle the packet.

Issues in Resource Allocation

- Taxonomy



Issues in Resource Allocation

- Taxonomy
  - Router-centric versus Host-centric
    - In a router-centric design, each router decides when packets are forwarded and selects which packets are forwarded and which are dropped, as well as for informs the hosts of they are allowed to send.
    - In a host-centric design, the end hosts observe the network conditions (e.g., how many packets they are successfully getting through the network) and adjust their behavior accordingly.

Issues in Resource Allocation

- Taxonomy
  - Reservation-based versus Feedback-based
    - In a reservation-based system, some entity asks the network for a certain amount of capacity to be allocated for a flow.
    - In a feedback-based approach, the end hosts begin sending data without first reserving any capacity and then adjust their sending rate according to the feedback they receive.
      - This feedback can either be *explicit (i.e., a congested router sends a"please* slow down" message to the host) or it can be *implicit (i.e., theend host adjusts its sending* rate according to the externally observablebehavior of the network, such as packet losses).

Issues in Resource Allocation

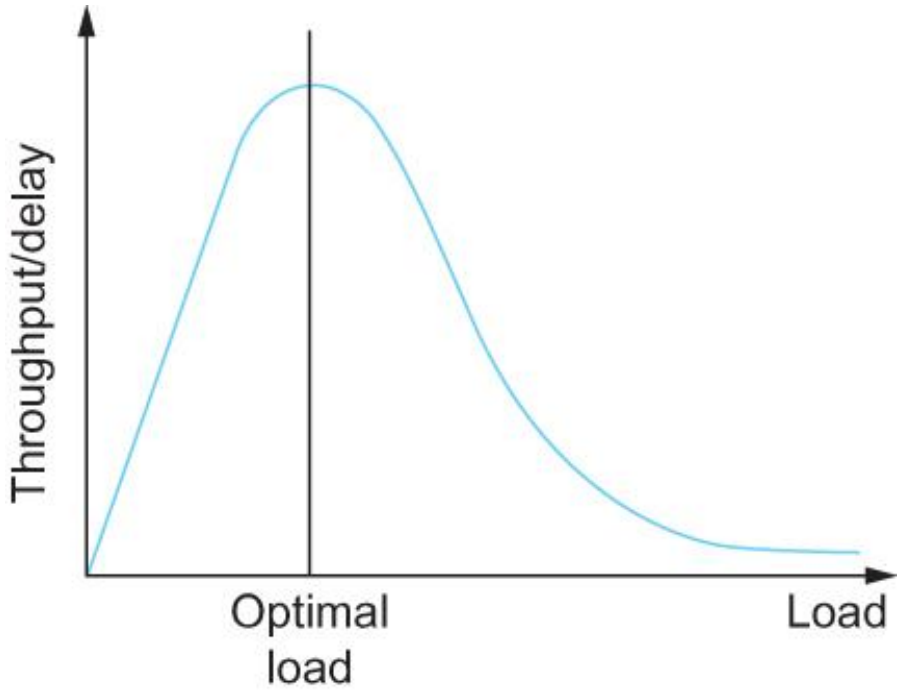
- Taxonomy
  - Window-based versus Rate-based
    - Window advertisement is used within the network to reserve buffer space.
    - Control sender's behavior using a rate, how many bit per second the receiver or network is able to absorb.

Issues in Resource Allocation

- Evaluation Criteria
  - Effective Resource Allocation
    - Two principal metrics of networking: maximize throughput and minimize delay.
    - One sure way to increase throughput is to allow as many packets into the network as possible, driving the utilization of all the links up to 100%.
    - The problem, increasing the number of packets also increases the length of the queues at each router. Longer queues, in turn, mean packets are delayed longer in the network

Issues in Resource Allocation

- Evaluation Criteria
  - Effective Resource Allocation



Ratio of throughput to delay as a function of load

Issues in Resource Allocation

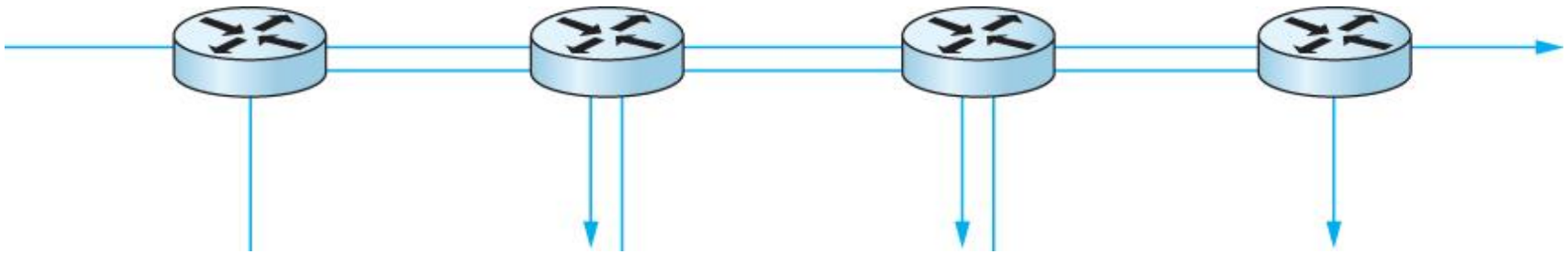
- Evaluation Criteria
  - Fair Resource Allocation
    - For example, a reservation-based resource allocation scheme provides an explicit way to create controlled unfairness.
    - With such a scheme, one might use reservations to enable a video stream to receive 1 Mbps across some link while a file transfer receives only 10 Kbps over the same link.

Issues in Resource Allocation

- Evaluation Criteria
  - Fair Resource Allocation
    - When several flows share a particular link, each flow should receive an equal share of the bandwidth.
    - This definition presumes that a *fair share of bandwidth means an equal share of bandwidth.*
    - But even in the absence of reservations, equal shares may not equate to fair shares.

Issues in Resource Allocation

- Evaluation Criteria
  - Fair Resource Allocation



One four-hop flow competing with three one-hop flows

Issues in Resource Allocation

- Evaluation Criteria
  - Fair Resource Allocation
    - Given a set of flow throughputs (x1, x2, . . . , xn) (measured inconsistent units such as bits/second),
    - Assuming that fair implies equal and that all paths are of equal length, the following function assigns a fairness index to the flows:

$$f(x_1, x_2, \ldots, x_n) = \frac{(\sum_{i=1}^{n} x_i)^2}{n \sum_{i=1}^{n} x_i^2}$$

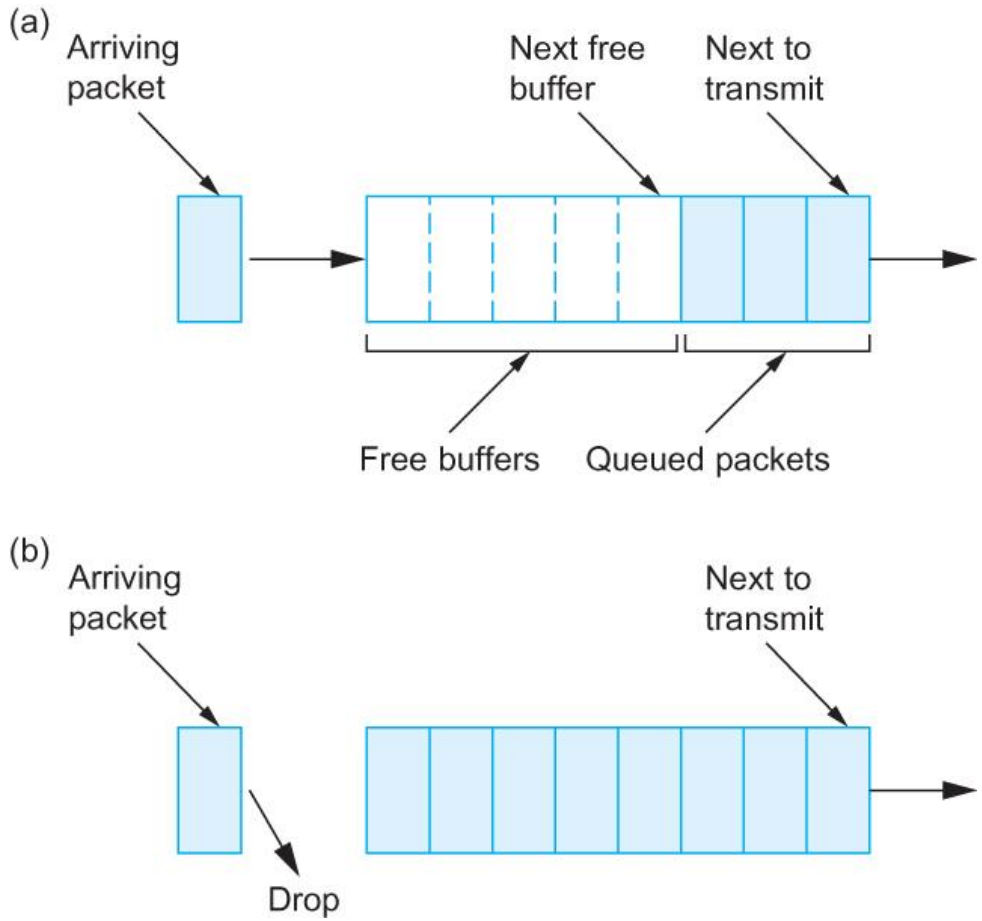
    - The fairness index always results in a number between 0 and 1, with 1representing greatest fairness.

QUEUING DISCIPLINES

Queuing Disciplines

- FIFO queuing / FCFS queuing:
  - The first packet that arrives at a router is the first packet to be transmitted.
  - If a packet arrives and the queue (buffer space) is full, then the router discards that packet.
  - This is done without regard to which flow the packet belongs to or how important the packet is. This is called *tail drop.*

Queuing Disciplines



(a) FIFO queuing; (b) tail drop at a FIFO queue.

Queuing Disciplines

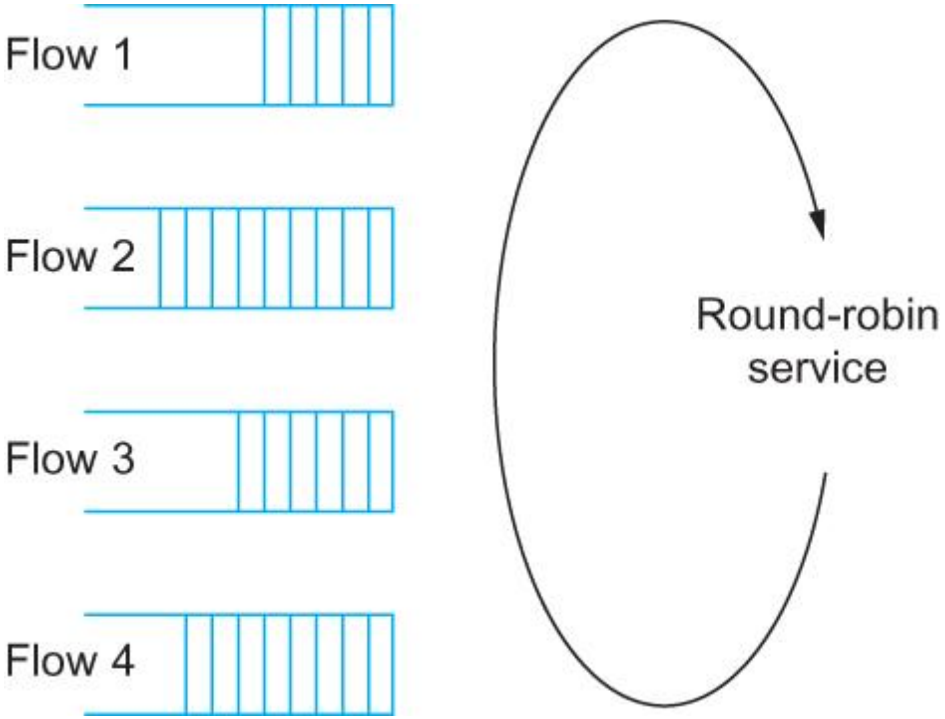
- A variation on basic FIFO queuing is priority queuing by marking each packet with a priority.
- The routers then implement multiple FIFO queues, one for each priority class.
- The router always transmits packets out of the highest-priority queue if that queue is non-empty before moving on to the next priority queue.
- Within each priority, packets are still managed in a FIFO manner.

Queuing Disciplines

- Fair Queuing
  - The main problem with FIFO queuing is that it doesnot separate packets according to the flow to whichthey belong.
  - Fair queuing (FQ) is an algorithm that maintains a separate queue for each flow currently being handled by the router.
  - The router then services these queues in a sort of round-robin,

Queuing Disciplines

- Fair Queuing



Round-robin service of four flows at a router

Queuing Disciplines

- Fair Queuing
  - The main complication with Fair Queuing is that the packets being processed at a router are not necessarily the same length.
  - To truly allocate the bandwidth of the outgoing link in a fair manner, it is necessary to take packet length into consideration.

Queuing Disciplines

- Fair Queuing
  - For example, if a router is managing two flows, one with 1000-byte packets and the other with 500-byte packets (perhaps because of fragmentation upstream from this router), then a simple round-robin servicing of packets from each flow's queue will give the first flow two thirds of the link's bandwidth and the second flow only one-third of its bandwidth.

Queuing Disciplines

- Fair Queuing
  - What's needed is bit-by-bit round-robin; that is, the transmit a bit from flow 1, then a bit from flow 2, and so on.
  - The FQ mechanism simulates this by determining when a given packet would finish being transmitted if it were being sent using bit-by-bit round-robin, and then uses this finishing time to sequence the packets for transmission.

Queuing Disciplines

- Fair Queuing
  - For this flow, let
    - P_i : denote the length of packet i
    - S_i: time when the router starts to transmit packet i
    - F_i: time when router finishes transmitting packet i
    - Clearly, $F_i = S_i + P_i$

Queuing Disciplines

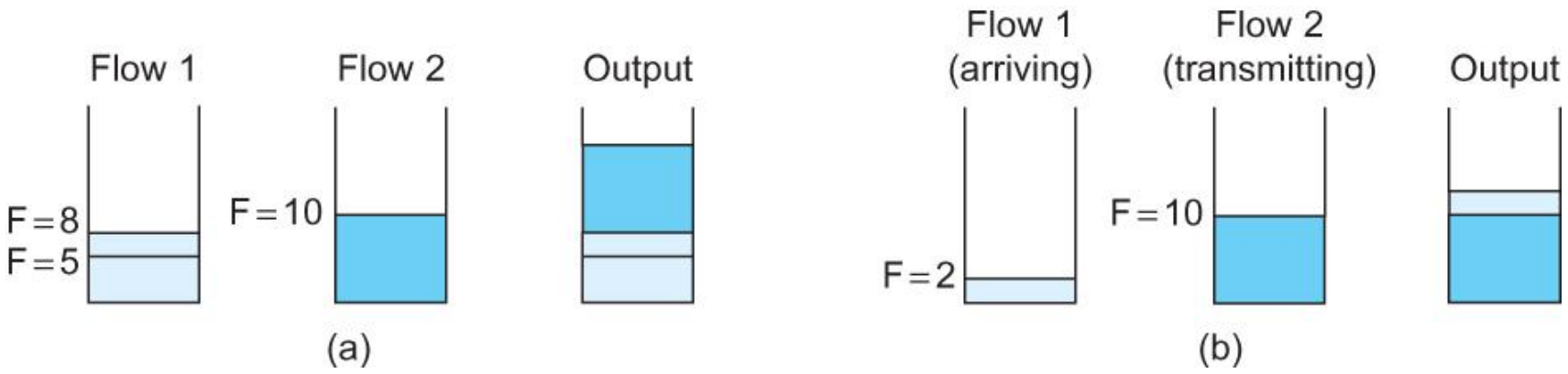
- Fair Queuing
  - When do we start transmitting packet *i*?
    - Depends on whether packet i arrived before or after the router finishes transmitting packet *i*-1 for the flow
  - Let A_i denote the time that packet *i* arrives at the router
  - Then $S_i = \max(F_{i-1}, A_i)$
  - $F_i = \max(F_{i-1}, A_i) + P_i$

Queuing Disciplines

- Fair Queuing
  - Next packet to transmit is always the packet that has the lowest F_i
  - The packet that should finish transmission before all others

Queuing Disciplines

- Fair Queuing



Example of fair queuing in action: (a) packets with earlier finishing times are sent first; (b) sending of a packet already in progress is completed

TCP CONGESTION CONTROL

TCP Congestion Control

- TCP congestion control was introduced into the Internet in the late 1980s by Van Jacobson, roughly eight years after the TCP/IP protocol stack had become operational.
- Immediately preceding this time, the Internet was suffering from congestion collapse

TCP Congestion Control

- The idea of TCP congestion control is for each source to determine how much capacity is available in the network, so that it knows how many packets it can safely have in transit.

TCP Congestion Control

- Once a source has some packets in transit, it uses the arrival of an ACK as a signal that one of its packets has left the network ➔ safe to insert a new packet into the network without congestion

- By using ACKs to pace the transmission of packets, TCP is said to be *self-clocking*.

TCP Congestion Control

- Additive Increase Multiplicative Decrease
  - TCP maintains a new variable, called *CongestionWindow*, which is used to limit how much data it is allowed in transit at a given time.
  - TCP is modified such that the maximum number of bytes of unacknowledged data allowed is now the minimum of the congestion window and the advertised window.

TCP Congestion Control

- Additive Increase Multiplicative Decrease
  - How does the source determine that the network is congested and that it should decrease the congestion window?
  - It is rare that a packet is dropped because of an error during transmission.
  - Therefore, TCP interprets timeouts as a sign of congestion and reduces the rate at which it is transmitting.

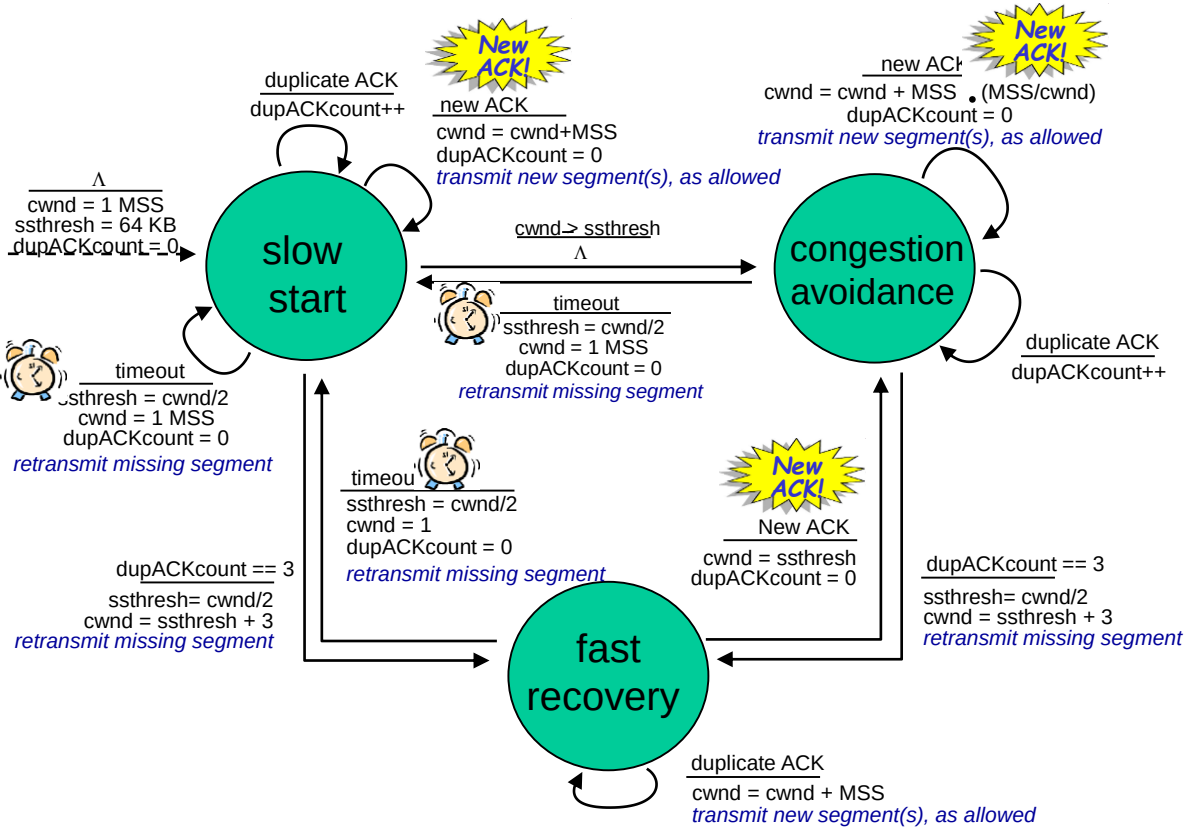
TCP Congestion Control

- Additive Increase Multiplicative Decrease
  - Each time a timeout occurs, the source sets CongestionWindow to half of its previous value.
  - This halving of the CongestionWindow for each timeout corresponds to the “multiplicative decrease” part of AIMD.

TCP Congestion Control

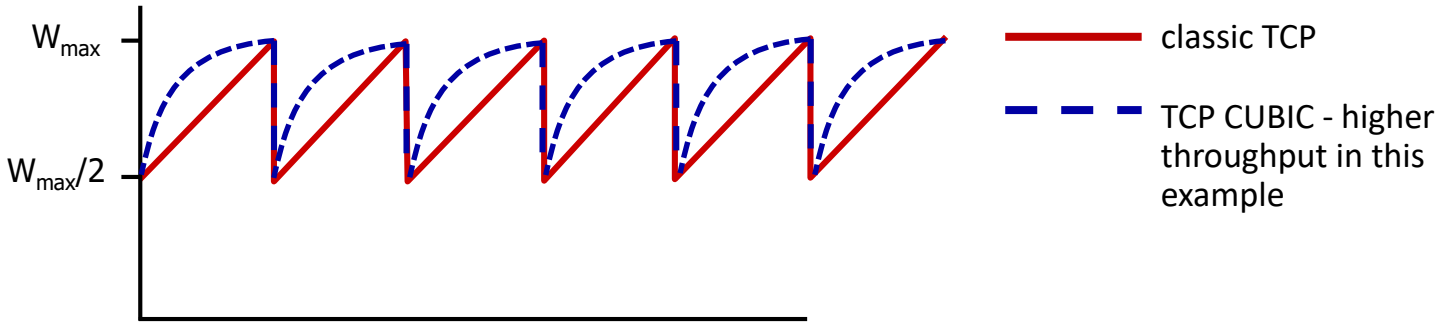
- Additive Increase Multiplicative Decrease
  - We also need to be able to increase the congestion window to take advantage of newly available capacity in the network.
  - This is the job of “additive increase” part of AIMD.

TCP congestion control



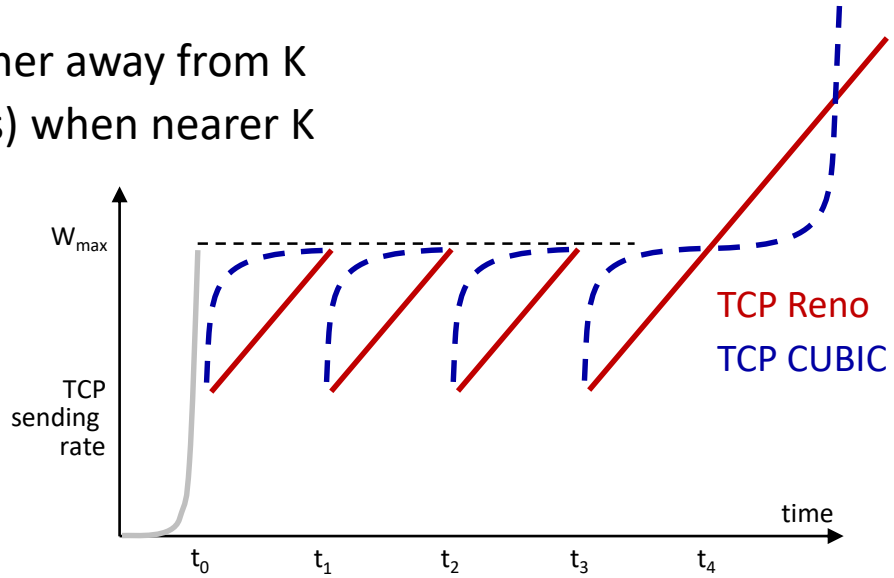
TCP CUBIC

- Is there a better way than AIMD to "probe" for usable bandwidth?
- Insight/intuition:
  - W_{max}: sending rate at which congestion loss was detected
  - congestion state of bottleneck link probably (?) hasn't changed much
  - after cutting rate/window in half on loss, initially ramp to W_{max} *faster*, but then approach W_{max} more *slowly*



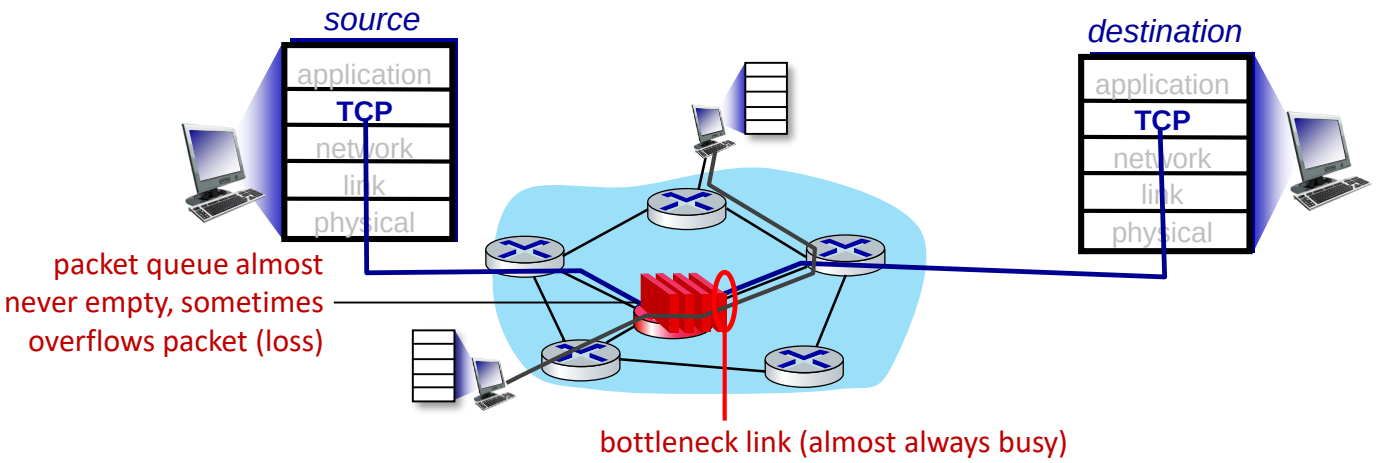
TCP CUBIC

- K: point in time when TCP window size will reach W_{max}
  - K itself is tunable
- increase W as a function of the *cube* of the distance between current time and K
  - larger increases when further away from K
  - smaller increases (cautious) when nearer K
- TCP CUBIC default in Linux, most popular TCP for popular Web servers (until ~2024)



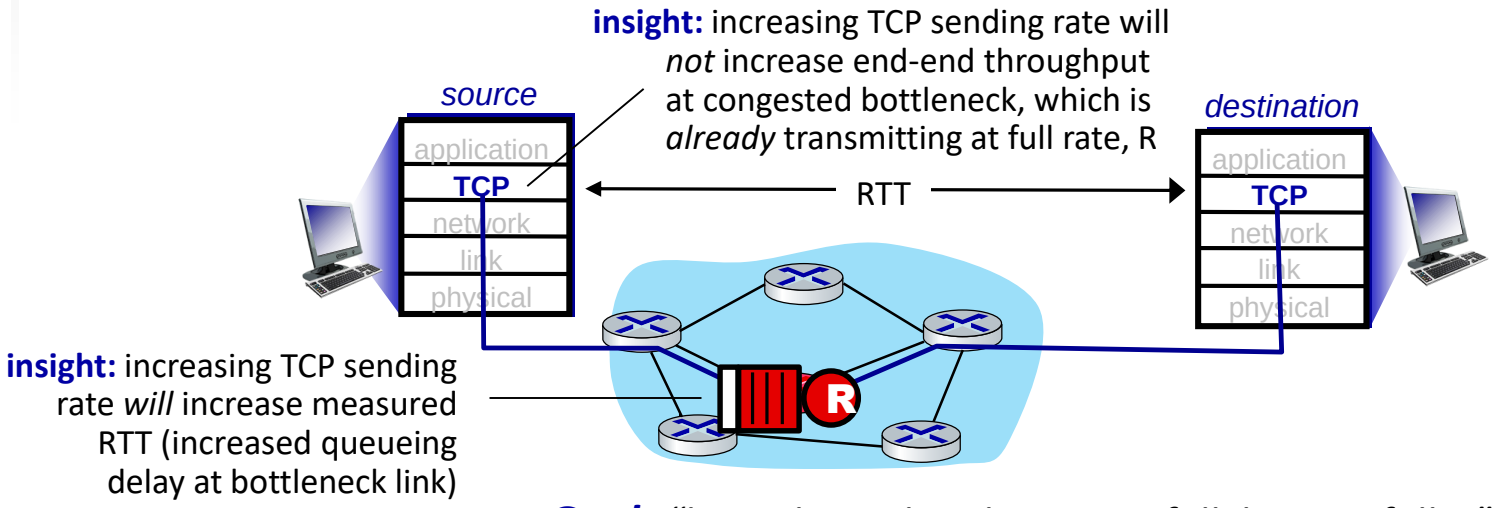
TCP and the congested "bottleneck link"

- TCP (classic, CUBIC) increase TCP's sending rate until packet loss occurs at some router's output: the *bottleneck link*



TCP and the congested "bottleneck link"

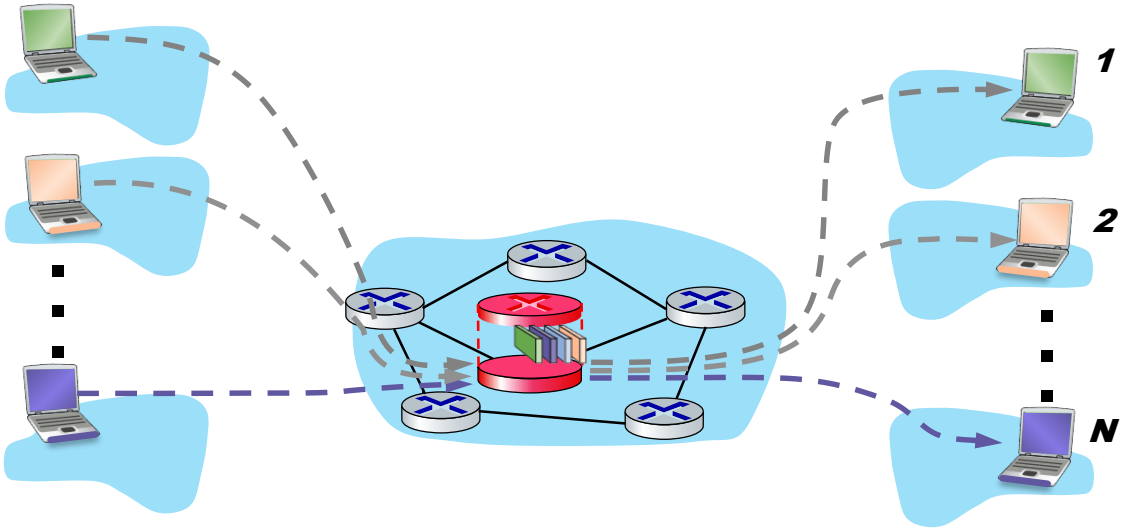
- TCP (classic, CUBIC) increase TCP's sending rate until packet loss occurs at some router's output: the *bottleneck link*
- understanding congestion: useful to focus on congested bottleneck link



Goal: *"keep the end-end pipe just full, but not fuller"*

TCP and the congested "bottleneck link"

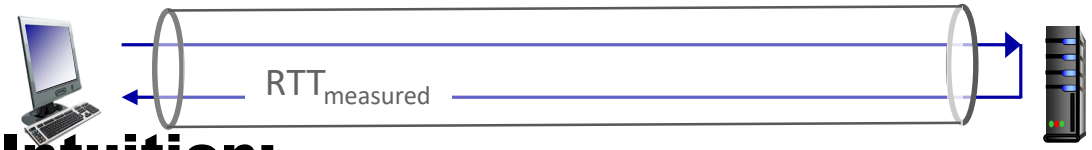
- multiple flows collectively congest bottleneck link:
  - each executing their own TCP congestion control
  - each of *N* flows should ideally receive throughput of *R/N*



Keeping the pipe just full enough

Keeping pipe "just full enough, but no fuller": keep bottleneck link busy transmitting, but avoid high delays/buffering

$$\text{measured throughput} = \frac{\text{\# bytes sent (inflight) in RTT interval}}{\text{RTT}}$$



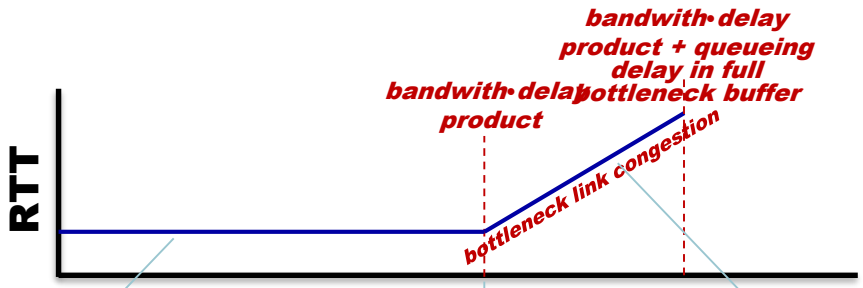
Intuition:

ideal amount of inflight data for a connection = connection's share of bottleneck_bandwidth • min_RTT

$n_{inflight}$

connection's "*bandwidth-delay product*"

Keeping the pipe just full, but no fuller



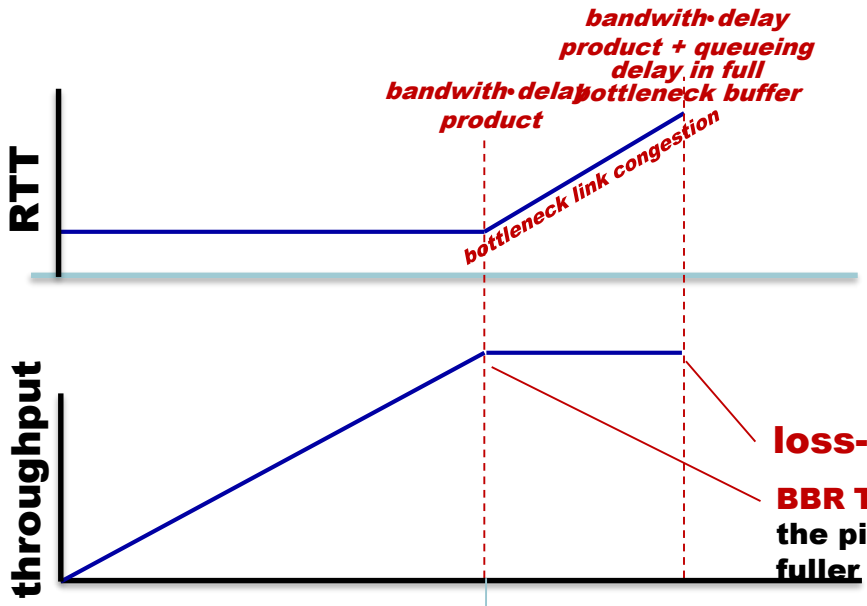
$n_{inflight}$: amount of in-flight data

increasing $n_{inflight}$ in this region will not increase RTT, since arrival rate of packet data to queue is less than transmission rate

Here, arrival rate of packet data to bottleneck link equals link transmission rate

Increasing $n_{inflight}$ in this region increases RTT since arrival rate of packet data to bottleneck link exceeds link transmission rate

Keeping the pipe just full, but no fuller



$n_{inflight}$: amount of in-flight data

Here, arrival rate of packet data to bottleneck link is equal to link transmission rate

Source-Based Algorithms: TCP Vegas

- Uses **RTT measurements** to detect early congestion
- Calculates:
  Expected rate=Window size/Base RTT
  Actual rate=Window size/Current RTT
- If Actual < Expected → congestion building → decrease rate
- Advantages: smoother throughput, fewer losses
- Drawback: can be outperformed by loss-based flows (Reno, CUBIC)

TCP BBR (Bottleneck Bandwidth and RTT)

- Developed by Google (2016)
- Models the network path: estimates **bottleneck bandwidth** and **RTT**
- Sends data at a rate ≈ *BottleneckBandwidth* × *RTT*
- Not loss-driven; focuses on maximizing delivery rate
- Adapts quickly to network changes
- Advantages: higher throughput, lower latency
- Limitation: coexistence issues with loss-based TCPs

BBR (Bottleneck Bandwidth and RTT)

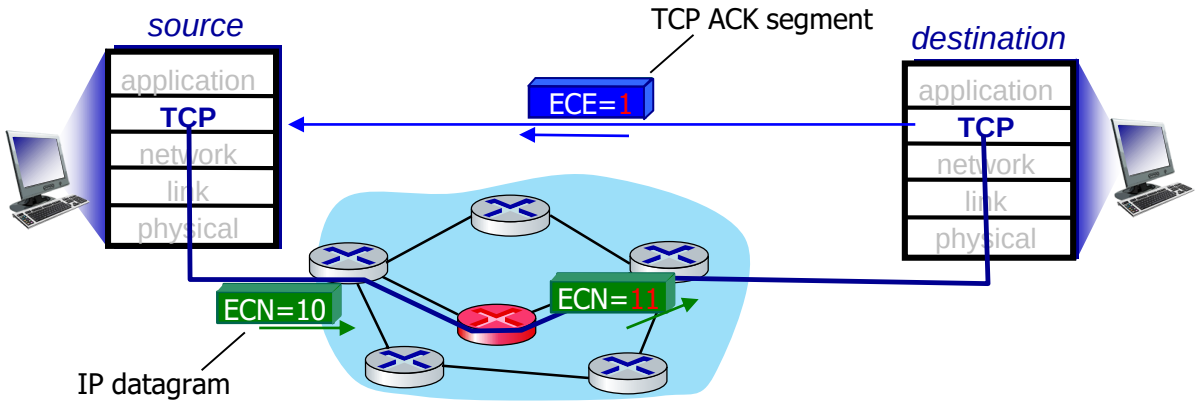
- regulates $n_{inflight}$
- at longer time intervals:
  - reduces $n_{inflight}$ to drain pipe, measure new *min_RTT*
- at shorter time intervals:

- *acceleration:* increases sending rate, $n_{inflight}$, until reaches throughput plateau, (saturating available per-flow link capacity)
- *cruising:* sends at the rate that network is delivering data, as evidenced through received ACKs
- *deceleration:* purposefully reduces $n_{inflight}$, decreasing queue pressure, looking for lower *min_RTT*

Explicit congestion notification (ECN)

TCP deployments often implement *network-assisted* congestion control:

- two bits in IP header (ToS field) marked *by network router* to indicate congestion
  - *policy* to determine marking chosen by network operator
- congestion indication carried to destination
- destination sets ECE bit on ACK segment to notify sender of congestion
- involves both IP (IP header ECN bit marking) and TCP (TCP header C,E bit marking)



DCTCP (Data Center TCP)

- Designed for **low-latency, high-throughput** data centers
- Uses **ECN** to provide fine-grained congestion feedback
- Switch marks packets when queue length > threshold K
- Sender computes fraction α of marked packets and updates window:
- Maintains high throughput with minimal queueing
- Requires ECN support and fair coexistence with classic TCP

DCTCP (Data Center TCP)

- **Mechanism:**
  - Adapts **ECN**: measures fraction of bytes encountering congestion
  - **Sender:** scales congestion window based on fraction of marked bytes
  - **Standard TCP** applies if actual packet loss occurs
- **Switch Operation:**
  - Packet arrives
  - Mark packet with CE bit if:

$$K > (RTT \times C)/7$$

- K = queue threshold, C = link rate

DCTCP (Data Center TCP)

- **Receiver Operation:**
- Maintain `CE` and `SeenCE` per flow
- On each packet:
  - CE=1 & SeenCE=False → SeenCE=True, send immediate ACK
  - CE=0 & SeenCE=True → SeenCE=False, send immediate ACK
  - Otherwise → continue delayed ACKs every n packets

$$K > (RTT \times C)/7$$

- Finally, sender computes the fraction of bytes that encountered congestion during the previous observation window
- total bytes transmitted / the bytes acknowledged with the ECE flag set

Final Words...

- DCTCP grows the congestion window in exactly the same way as the standard algorithm
- DCTCP reduces the window in proportion to how many bytes encountered congestion during the last observation window.